

claude-windows / 144a31b7-7962-4fee-9d6f-8a7854daa63f

Metadata

```
- Source: `claude-windows`  
- Kind: `claude`  
- Source file: `C:\Users\avidu\.claude\projects\C--Users-avidu-Projects-badminton-highlight-indexer\144a31b7-7962-4fee-9d6f-8a7854daa63f\subagents\workflows\wf_1c5f1f5c-9ba\agent-a2ea8796a80e3db72.jsonl`  
- SHA-256: `611469311ac213c49a442d336f7a982780a696cbab56d61334cf8bbcc3d78ba7`  
- Source modified: `2026-06-20T08:02:15+00:00`  
- Imported at: `2026-07-05T16:48:28+00:00`  
- project: `wf_1c5f1f5c-9ba`  
- session_id: `144a31b7-7962-4fee-9d6f-8a7854daa63f`
```

Transcript

1. user (2026-06-20T08:00:25.394Z)

You are reading the badminton-highlight-indexer repo (your cwd). Map the engine's EXISTING HTTP API. Start at backend/main.py and backend/api/uploads.py and follow all routers. For each endpoint report method, path, purpose, response shape, and auth requirement. Pay special attention to anything that: lists videos, lists rallies/segments for a video, serves a video or proxy or frame, builds/triggers a reel, downloads a reel, accepts corrections/labels. Also determine the AUTH model (token? DEV_USER_EMAIL? none?) ? read docs/HOSTING_PLAN.md and docs/UI_ALPHA_EXECUTION_PLAN.md for the intended auth/hosting. Then list GAPS: concretely, what does an interactive per-rally-selection UI need from the API that does NOT exist today (e.g. GET rallies for a video as JSON, per-rally thumbnail, POST a custom selection -> reel). Quote path:line. Do not invent endpoints. Return the schema.

2. assistant (2026-06-20T08:00:28.958Z)

I'll map the engine's HTTP API. Let me start by reading the entry points and discovering all routers.

3. user (2026-06-20T08:00:30.484Z)

```
1      import os  
2      import sys  
3      import argparse  
4      import logging  
5      import uuid  
6      import uvicorn  
7      from dotenv import load_dotenv  
8  
9      load_dotenv()  
10  
11     # Centralized logging setup for the indexer (hot paths use logger, CLI output uses print  
for direct user-facing messages).  
12     logging.basicConfig(  
13         level=logging.INFO,  
14         format='%(asctime)s [%(levelname)s] %(name)s: %(message)s',  
15         datefmt='%H:%M:%S'  
16     )  
17     logger = logging.getLogger(__name__)  
18     from fastapi import FastAPI, HTTPException  
19     from fastapi.middleware.cors import CORSMiddleware  
20     from pydantic import BaseModel  
21     from typing import List, Dict, Any, Optional  
22  
23     from fastapi.staticfiles import StaticFiles  
24     from fastapi.responses import FileResponse, HTMLResponse, JSONResponse  
25     from backend.pipeline.validators.base import registry
```

```

26     from backend.infrastructure.database import Database
27     from backend.pipeline.segmenters.base import segmenter_registry
28     from backend.pipeline.stitcher.compiler import VideoCompiler
29     from backend.utils.hashing import compute_video_id # canonical file-identity hashing
30     from backend.utils.paths import resolve_under_root, safe_output_filename,
validate_input_path
31     from backend.config import load_config as load_app_config, AppConfig, StitchingConfig #
new typed config
32     from backend.results import Failure # standardized error/result contract
33     from backend.reporting import emit_indexer_report
34
35     app = FastAPI(title="Sports Highlight Indexer API")
36
37     # Enable CORS for the local web interface. allow_origins='' with allow_credentials=True
is
38     # rejected by browsers per the fetch spec and is an unsafe default to carry into the
39     # auth/tenancy work; this tool uses no cookies, so credentials stay off.
40     app.add_middleware(
41         CORSMiddleware,
42         allow_origins=["*"],
43         allow_credentials=False,
44         allow_methods=["*"],
45         allow_headers=["*"],
46     )
47
48     # Serves static frontend files directly (now under api/ per approved structure)
49     os.makedirs("backend/api/static", exist_ok=True)
50     app.mount("/static", StaticFiles(directory="backend/api/static"), name="static")
51
52     @app.get("/", response_class=HTMLResponse)
53     def serve_index():
54         index_path = "backend/api/static/index.html"
55         if os.path.exists(index_path):
56             with open(index_path, "r", encoding="utf-8") as f:
57                 return f.read()
58         return "<h3>Sports Highlight Indexer Server is Running. Please create index.html in
backend/api/static/</h3>"
59
60     # Global variables initialized at startup
61     db_instance: Optional[Database] = None
62     global_config: Optional[AppConfig] = None # now typed (Pydantic model)
63
64     # Async job worker ? drain/wake loop (DEFERRED_HARDENING_PLAN #16, EXECUTION_POLICY.md
P3)
65     # now lives in backend.api.job_worker. Re-exported here so the names stay addressable on
66     # `backend.main` (`JobWaiting`, `_drain_due_jobs`, `_arm_wake_timer`, `_get_executor`,
67     # `_run_claimed_job`, `_MAX_DRAIN_WAKE_SEC`, ...). The worker resolves these seams
THROUGH
68     # this module object at call-time, so `monkeypatch.setattr(backend.main, ...)` still
69     # intercepts the inter-function calls (a parked job arms the PATCHED `_arm_wake_timer`,
70     # so no real daemon timer leaks in tests). Behavior is unchanged by the extraction.
71     from backend.api.job_worker import ( # noqa: F401 (re-exports: names stay addressable
on backend.main)
72         JobWaiting,
73         _arm_wake_timer,
74         _drain_due_jobs,
75         _get_executor,
76         _job_to_request,
77         _MAX_DRAIN_WAKE_SEC,
78         _run_claimed_job,
79         _schedule_next_drain_if_waiting,
80     )
81
82     # Legacy thin wrapper for any remaining direct calls during transition.
83     # New code should import load_app_config / AppConfig from backend.config directly.

```

```

84     def load_config(config_path: str = "config.json") -> Dict[str, Any]:
85         cfg = load_app_config(config_path)
86         return cfg.model_dump(mode="json")
87
88     # --- API Models ---
89     class ProcessRequest(BaseModel):
90         video_path: str
91         player_pool: Optional[List[str]] = None
92         max_frames: Optional[int] = None
93         segmenter: Optional[str] = None
94
95     class QueryRequest(BaseModel):
96         video_id: str
97         server: Optional[str] = None
98         receiver: Optional[str] = None
99         duration_min: Optional[float] = None
100         event_type: Optional[str] = None
101
102     class CompileRequest(BaseModel):
103         video_id: str
104         segment_ids: List[int]
105         output_filename: str
106
107     def _finalize_reingest(video_id: str, segments, play_wm: int, cand_wm: int) -> None:
108         """Commit a (re)segmentation with destroy-AFTER-confirm semantics.
109
110         On a non-empty run, drop the PRIOR rows (id <= the pre-run watermark) so only the
111         fresh segments remain. On an empty run, drop the NEW partial rows (id > watermark)
112         and
113         keep the prior good results intact ? a failed/empty re-run never destroys prior
114         data.
115         """
116         if segments:
117             db_instance.prune_segments(video_id, play_upto=play_wm, cand_upto=cand_wm)
118         else:
119             db_instance.prune_segments(video_id, play_after=play_wm, cand_after=cand_wm)
120
121     # --- API Endpoints ---
122     @app.get("/api/config")
123     def get_config():
124         return global_config
125
126     def _execute_processing(req: ProcessRequest, progress=None) -> Dict[str, Any]:
127         """The full hash ? validate ? segment ? report pipeline for one video.
128
129         This is the former synchronous /api/process body, unchanged in behavior, now run on
130         the job worker thread (DEFERRED_HARDENING_PLAN #16). Returns the same response dict
131         the sync endpoint used to return (UI compatibility); the per-video observability
132         report is still emitted in `finally:` and grafted as response["reports"].
133
134         `progress` is an optional callable(stage: str) used to surface coarse job progress.
135         Raises on unexpected internal errors ? the caller records those as a job failure
136         (after this function has already restored the pre-run segments, see below).
137         """
138         notify = progress or (lambda stage: None)
139
140         notify("hashing")
141         video_id = compute_video_id(req.video_path)
142         was_reingest = db_instance.get_video(video_id) is not None
143
144         # 1. Run pluggable validators (with-context variant surfaces ffprobe_meta for the
145         report)
146         notify("validating")
147         logger.info(f"Running validations for {req.video_path}...")

```

```

146         val_results, val_context = registry.run_all_with_context(req.video_path,
global_config)
147         failures = [r for r in val_results if not r.passed]
148
149         # typed AppConfig: attribute access for the default segmenter (with safe fallback)
150         default_seg = getattr(getattr(global_config, "indexing", None), "default_segmenter",
"motion")
151         seg_name = req.segmenter or default_seg
152         segmenter_actual = None
153         segmentation_ran = False
154         response: Optional[Dict[str, Any]] = None
155         try:
156             if failures:
157                 # add_video is an UPSERT (no cascade) ? a failed re-validation only updates
158                 # status; it no longer destroys the video's prior good segments.
159                 db_instance.add_video(video_id, req.video_path, "failed", [r.model_dump()
for r in val_results])
160                 response = {
161                     "video_id": video_id,
162                     "status": "failed",
163                     "message": "Video failed validation checks.",
164                     "results": [r.model_dump() for r in val_results],
165                 }
166                 return response
167
168             # 2. Run segmenter & indexer
169             logger.info("[API] Video passed checks. Segmenting and indexing...")
170             db_instance.add_video(video_id, req.video_path, "processed", [r.model_dump() for
r in val_results])
171
172             segmenter_cls = segmenter_registry.get(seg_name)
173             if not segmenter_cls:
174                 # Submit-time validation already 400s unknown names; this is the defensive
175                 # in-worker path (e.g. config default pointing at an unregistered
segmenter).
176                 available = list(segmenter_registry.list_available().keys())
177                 response = {
178                     "video_id": video_id,
179                     "status": "failed",
180                     "message": f"Segmenter '{seg_name}' not found. Available segmenters:
{available}",
181                     "results": [r.model_dump() for r in val_results],
182                     "play_segments": [],
183                 }
184                 return response
185
186             logger.info(f"[API] Using segmenter: {seg_name}")
187             notify(f"segmenting ({seg_name})")
188             segmenter_actual = seg_name
189             # Destroy-AFTER-confirm: remember the pre-run rows, then keep new + drop old
only
190             # if the run yields segments; otherwise drop the new partial rows and keep the
old.
191             play_wm, cand_wm = db_instance.segment_watermarks(video_id)
192             segmenter = segmenter_cls(db_instance, global_config)
193             try:
194                 result = segmenter.process_video(video_id, req.video_path, req.player_pool,
req.max_frames)
195             except Exception:
196                 # A raising segmenter must not leave half-written rows: restore the pre-run
197                 # state (same destroy-after-confirm contract as the Failure branch), then
let
198                 # the job worker record the failure.
199                 db_instance.prune_segments(video_id, play_after=play_wm, cand_after=cand_wm)
200                 raise

```

```

201         segmentation_ran = True
202
203     if isinstance(result, Failure):
204         logger.error(f"[API] Segmenter failed: {result.message}")
205         db_instance.prune_segments(video_id, play_after=play_wm, cand_after=cand_wm)
206         response = {
207             "video_id": video_id,
208             "status": "failed",
209             "message": f"Segmenter '{seg_name}' failed: {result.message}",
210             "results": [r.model_dump() for r in val_results],
211             "play_segments": [],
212         }
213         return response
214
215     segments = result.value if hasattr(result, "value") else result
216     notify("finalizing")
217     _finalize_reingest(video_id, segments, play_wm, cand_wm)
218
219     response = {
220         "video_id": video_id,
221         "status": "success",
222         "message": f"Successfully indexed {len(segments)} play segments using
223 '{seg_name}' segmenter.",
224         "results": [r.model_dump() for r in val_results],
225         "play_segments": segments,
226     }
227     return response
228 finally:
229     # Always emit the per-video observability report (next to the video, or to
230     # report.output_dir). Never raises. Surface the paths in the response.
231     paths = emit_indexer_report(
232         db=db_instance, video_id=video_id, video_path=req.video_path,
233         config=global_config,
234         val_results=val_results, val_context=val_context,
235         segmenter_requested=req.segmenter, segmenter_actual=segmenter_actual,
236         was_reingest=was_reingest, segmentation_ran=segmentation_ran,
237     )
238     if paths and isinstance(response, dict):
239         response["reports"] = paths
240
241 @app.post("/api/process", status_code=202)
242 def process_video(req: ProcessRequest):
243     """Submit a processing job (202 + job_id). Poll GET /api/jobs/{job_id} for state.
244
245     Fast-fail checks (bad path, missing file, unknown segmenter) happen here so the
246     client gets an immediate 4xx; everything slow ? including MD5 hashing of the file ?
247     runs on the worker (DEFERRED_HARDENING_PLAN #16). The row is created 'queued'; the
248     worker drains it via `claim_due_job` (EXECUTION_POLICY.md P3 self-scheduling loop).
249     """
250     allowed_root = getattr(getattr(global_config, "security", None),
251 "allowed_video_root", None)
252     try:
253         validate_input_path(req.video_path, allowed_root=allowed_root)
254     except ValueError as e:
255         raise HTTPException(status_code=400, detail=str(e)) from e
256     if not os.path.exists(req.video_path):
257         raise HTTPException(status_code=404, detail=f"Video file not found at
258 '{req.video_path}'")
259     if req.segmenter and not segmenter_registry.get(req.segmenter):
260         available = list(segmenter_registry.list_available().keys())
261         raise HTTPException(
262             status_code=400,
263             detail=f"Segmenter '{req.segmenter}' not found. Available segmenters:
264 {available}"

```

```

261         )
262
263     job_id = uuid.uuid4().hex
264     if not db_instance.create_job(job_id, req.video_path, req.segmenter,
265                                 req.player_pool, req.max_frames):
266         raise HTTPException(status_code=500, detail="Failed to persist job record.")
267     _get_executor().submit(_drain_due_jobs)
268     return JSONResponse(
269         status_code=202,
270         content={"job_id": job_id, "status": "queued", "poll": f"/api/jobs/{job_id}"},
271     )
272
273
274 @app.get("/api/jobs/{job_id}")
275 def get_job(job_id: str):
276     """Job state: {status, progress, result, reports}. `status` = execution state
277     (queued|running|done|failed); the pipeline verdict is result["status"]."""
278     job = db_instance.get_job(job_id)
279     if not job:
280         raise HTTPException(status_code=404, detail=f"Unknown job_id '{job_id}'")
281     result = job.get("result")
282     return {
283         "job_id": job["id"],
284         "status": job["status"],
285         "progress": job.get("progress"),
286         "video_id": job.get("video_id"),
287         "error": job.get("error"),
288         "result": result,
289         "reports": (result or {}).get("reports"),
290         "created_at": job.get("created_at"),
291         "started_at": job.get("started_at"),
292         "finished_at": job.get("finished_at"),
293     }
294
295 # --- Chunked upload (B2, PR-2) -----
296 # The chunked-upload island (in-process registry + helpers + the 4 endpoints) now lives
297 in
298 # backend.api.uploads as a self-contained APIRouter. It resolves the live startup config
299 via
300 # backend.main.global_config, so behavior ? route paths, status codes, body-size limits,
301 # sequential-part logic, and abort cleanup ? is unchanged.
302 from backend.api import uploads as uploads_api
303
304 app.include_router(uploads_api.router)
305
306
307 # --- Highlight download (B3, PR-2) -----
308
309 @app.get("/api/highlights/{video_id}/{filename}")
310 def download_highlight(video_id: str, filename: str):
311     """Stream a compiled highlight reel ? `filename` is the bare name given to
312     /api/compile
313     (which only writes into output_dir; the browser previously got back an unreachable
314     server-local path)."""
315     try:
316         name = safe_output_filename(filename)
317     except ValueError as e:
318         raise HTTPException(status_code=400, detail=str(e)) from e
319     if not db_instance.get_video(video_id):
320         raise HTTPException(status_code=404, detail="Indexed video record not found.")
321     output_dir = getattr(getattr(global_config, "output", None), "output_dir", "output")
322     or "output"
323     path = os.path.join(output_dir, name)
324     try:
325         path = resolve_under_root(path, output_dir)

```

```

322     except ValueError as e:
323         raise HTTPException(status_code=400, detail=str(e)) from e
324     if not os.path.isfile(path):
325         raise HTTPException(status_code=404,
326                             detail=f"No compiled file named '{name}'. Compile first via
/api/compile.")
327     media = "video/quicktime" if name.lower().endswith(".mov") else "video/mp4"
328     return FileResponse(path, media_type=media, filename=name)
329
330
331 @app.post("/api/query")
332 def query_play_segments(req: QueryRequest):
333     filters = {
334         "server": req.server,
335         "receiver": req.receiver,
336         "duration_min": req.duration_min,
337         "event_type": req.event_type
338     }
339     segments = db_instance.query_play_segments(req.video_id, filters)
340     return {"play_segments": segments}
341
342 @app.post("/api/compile")
343 def compile_highlights(req: CompileRequest):
344     video = db_instance.get_video(req.video_id)
345     if not video:
346         raise HTTPException(status_code=404, detail="Indexed video record not found.")
347
348     # Retrieve matching play segments from db
349     all_segments = db_instance.query_play_segments(req.video_id, {})
350     matched_segments = [s for s in all_segments if s["id"] in req.segment_ids]
351
352     if not matched_segments:
353         raise HTTPException(status_code=400, detail="No matching play segments found to
stitch.")
354
355     # Reject path traversal / absolute output names (os.path.join would otherwise let an
356     # absolute or '..'-laden output_filename write anywhere on disk).
357     try:
358         out_name = safe_output_filename(req.output_filename)
359     except ValueError as e:
360         raise HTTPException(status_code=400, detail=str(e)) from e
361
362     # The configured shot-count floor (stitching.min_shot_count) applies on the API
363     # path too ? parity with scripts/run_process_and_stitch.py. Segments WITHOUT
shot_count
364     # metadata are exempt: the floor filters known counts only, so explicitly
365     # requested manual/legacy segments can't silently vanish under the default floor.
366     stitching = getattr(global_config, "stitching", None) or StitchingConfig()
367     kept = []
368     for s in matched_segments:
369         meta = s.get("metadata") or {}
370         sc = meta.get("shot_count") if isinstance(meta, dict) else None
371         try:
372             if sc is not None and int(sc) < stitching.min_shot_count:
373                 continue
374             except (TypeError, ValueError):
375                 pass # unparseable count -> treat as unknown, keep
376             kept.append(s)
377     if len(kept) < len(matched_segments):
378         logger.info(f"[compile] stitching.min_shot_count={stitching.min_shot_count}:
dropped "
379                     f"{len(matched_segments) - len(kept)}/{len(matched_segments)}
segments below the floor.")
380     if not kept:
381         raise HTTPException(status_code=400,

```

```

382                 detail=f"All {len(matched_segments)} matching segments fall
below "
383
384 f"stitching.min_shot_count={stitching.min_shot_count}.")
385
386     # Extract start/end time pairs
387     time_pairs = [(s["start_time"], s["end_time"]) for s in kept]
388
389     # Run compiler with the configured stitching paddings + optional branding watermark
390     output_dir = getattr(getattr(global_config, "output", None), "output_dir", "output")
391     or "output"
392     compiler = VideoCompiler(output_dir, begin_padding=stitching.begin_padding,
393                             end_padding=stitching.end_padding,
394                             watermark=stitching.watermark)
395
396     try:
397         out_path = compiler.compile_highlights(video["filepath"], time_pairs, out_name)
398         return {"status": "success", "filepath": out_path}
399     except Exception as e:
400         raise HTTPException(status_code=500, detail=f"Highlight stitching failed:
{str(e)}") from e
401
402 # --- CLI Debug Utility & Orchestration ---
403 def run_cli_diagnose_clip(video_path: str, timestamp: float):
404     """CLI debugging utility to examine segment properties around a timestamp."""
405     print("=" * 60)
406     print(f"DEBUGGING VIDEO CLIP: {video_path} at {timestamp}s")
407     print("=" * 60)
408
409     cfg = load_config() # legacy wrapper still works, returns dict for now
410
411     # 1. Validation check diagnostics
412     print("[DIAGNOSTIC] Running validation framework...")
413     results = registry.run_all(video_path, cfg)
414     for r in results:
415         status_symbol = "[PASS]" if r.passed else "[FAIL]"
416         print(f"    {status_symbol} {r.validator_name}: {r.message}")
417         if r.details:
418             print(f"        Details: {r.details}")
419
420     # 2. Local segment analysis around the timestamp (sampling +/- 5 seconds)
421     print("-" * 60)
422     print("[DIAGNOSTIC] Analyzing local motion around the timestamp...")
423     import cv2
424     cap = cv2.VideoCapture(video_path)
425     if not cap.isOpened():
426         print("[FAIL] OpenCV could not open video.")
427         return
428
429     fps = cap.get(cv2.CAP_PROP_FPS)
430     total_frames = cap.get(cv2.CAP_PROP_FRAME_COUNT)
431
432     start_frame = max(0, int((timestamp - 3.0) * fps))
433     end_frame = min(int(total_frames), int((timestamp + 3.0) * fps))
434
435     # Measure frame-by-frame changes in sample region
436     cap.set(cv2.CAP_PROP_POS_FRAMES, start_frame)
437     prev_gray = None
438     motions = []
439
440     for _ in range(start_frame, end_frame):
441         ret, frame = cap.read()
442         if not ret:
443             break
444         gray = cv2.cvtColor(frame, cv2.COLOR_BGR2GRAY)
445         gray = cv2.GaussianBlur(gray, (21, 21), 0)

```



```

442
443         if prev_gray is not None:
444             diff = cv2.absdiff(prev_gray, gray)
445             _, thresh = cv2.threshold(diff, 25, 255, cv2.THRESH_BINARY)
446             motion_ratio = cv2.countNonZero(thresh) / (thresh.shape[0] *
thresh.shape[1])
447             motions.append(motion_ratio)
448             prev_gray = gray
449
450     cap.release()
451
452     if motions:
453         avg_motion = sum(motions) / len(motions)
454         # Support both legacy dict (from wrapper) and future direct model
455         if hasattr(cfg, "indexing"):
456             threshold = getattr(cfg.indexing, "motion_threshold", 0.08)
457         else:
458             threshold = cfg.get("indexing", {}).get("motion_threshold", 0.08) # type:
ignore[attr-defined]
459         print(f" Sample Frame Count: {len(motions)}")
460         print(f" Average motion index around timestamp: {round(avg_motion, 4)}
(threshold: {threshold})")
461         if avg_motion > threshold:
462             print(" Result: [ACTIVE PLAY] High motion detected. Segment classifies as
active play.")
463         else:
464             print(" Result: [DEAD TIME] Idle movement. Segment classified as dead
time.")
465     else:
466         print(" [ERROR] No visual frames were extracted in the sample range.")
467
468     print("=" * 60)
469
470     def main():
471         parser = argparse.ArgumentParser(description="Sports Highlight Indexer
Orchestrator")
472         parser.add_argument("--video-path", help="Local path to the MP4 sports video file")
473         parser.add_argument("--output-dir", default="output", help="Directory where
processed clips/highlights are saved")
474         parser.add_argument("--setup", action="store_true", help="Launch dependency checker
and setup utility")
475         parser.add_argument("--debug-clip", type=float, help="Timestamp (in seconds) of a
video clip to run detailed diagnostics on")
476         parser.add_argument("--server", action="store_true", help="Run the FastAPI local API
server")
477         parser.add_argument("--port", type=int, default=8000, help="Port to run the FastAPI
server on")
478         parser.add_argument("--max-frames", type=int, help="Limit processing to the first N
sub-sampled frames for debugging")
479
480         available_segmenters = list(segmenter_registry.list_available().keys())
481         parser.add_argument("--segmenter", choices=available_segmenters, help=f"Choose rally
detection segmenter. Available: {available_segmenters}")
482         parser.add_argument("--trim-start", type=float, help="Start time in seconds for the
debug trim segment feature")
483         parser.add_argument("--trim-end", type=float, help="End time in seconds for the
debug trim segment feature")
484         parser.add_argument("--trim-output", help="Output filename for the debug trimmed
segment (saves in output-dir unless absolute path)")
485
486         args = parser.parse_args()
487
488         if args.setup:
489             # Invoke the diagnostics/bootstrap script (renamed from the old root
490             # setup.py to scripts/doctor.py so it no longer shadows the PEP 517

```

```

491         # build system). Resolve its path from this file so `indexer --setup`
492         # works regardless of the caller's cwd.
493         import subprocess
494         doctor_path = os.path.join(
495             os.path.dirname(os.path.dirname(os.path.abspath(__file__))),
496             "scripts", "doctor.py",
497         )
498         print(f"[INFO] Invoking diagnostics: {doctor_path} ...")
499         subprocess.run([sys.executable, doctor_path])
500         return
501
502     if args.trim_start is not None and args.trim_end is not None:
503         if not args.video_path:
504             print("[ERROR] Please provide --video-path to extract a segment.")
505             return
506
507         # Determine output path
508         out_filename = args.trim_output or "trimmed_segment.mp4"
509         if os.path.isabs(out_filename):
510             out_path = out_filename
511             out_dir = os.path.dirname(out_path)
512             out_name = os.path.basename(out_path)
513         else:
514             out_dir = args.output_dir
515             out_path = os.path.join(out_dir, out_filename)
516             out_name = out_filename
517
518         os.makedirs(out_dir, exist_ok=True)
519         print(f"[CLI] Extracting debug segment from {args.trim_start}s to
520 {args.trim_end}s from {args.video_path}...")
521
522         # global_config isn't initialized this early; load the typed config so the trim
523         # clip carries the same configured stitching paddings + watermark as a real
524         # highlight (watermark default-on).
525         stitching = load_app_config().stitching
526         compiler = VideoCompiler(out_dir, begin_padding=stitching.begin_padding,
527 watermark=stitching.watermark)
528         try:
529             res_path = compiler.compile_highlights(args.video_path, [(args.trim_start,
530 args.trim_end)], out_name)
531             print(f"[SUCCESS] Trimmed segment saved to: {res_path}")
532         except Exception as e:
533             print(f"[ERROR] Failed to compile trimmed segment: {e}")
534             return
535
536     if args.debug_clip is not None:
537         if not args.video_path:
538             print("[ERROR] Please provide --video-path to debug a specific clip.")
539             return
540         run_cli_diagnose_clip(args.video_path, args.debug_clip)
541         return
542
543     # Initialize Global Config and DB
544     global global_config, db_instance
545     global_config = load_app_config() # returns typed AppConfig
546
547     # The CLI --output-dir overrides the configured output directory. It now lives
548     # on the typed model (OutputConfig.output_dir), so every consumer ? including
549     # /api/compile ? reads the same value instead of a write-only runtime hack.
550     global_config.output.output_dir = args.output_dir
551     os.makedirs(global_config.output.output_dir, exist_ok=True)
552
553     db_path = os.path.join(global_config.output.output_dir,
554 global_config.output.db_name)

```

```

552     db_instance = Database(db_path)
553
554     # Crash recovery (#16): jobs left queued/running by a previous process are dead ?
555     # the executor is in-process. Mark them failed so pollers see a terminal state.
556     swept = db_instance.fail_stale_jobs()
557     if swept:
558         logger.warning(f"[JOBS] Swept {swept} stale job(s) from a previous run ?
failed.")
559
560     # CLI processing mode (no web UI needed)
561     if args.video_path and not args.server:
562         print(f"[CLI] Processing video file: {args.video_path}")
563         if not os.path.exists(args.video_path):
564             print(f"[FAIL] Video file not found: {args.video_path}")
565             return
566
567         video_id = compute_video_id(args.video_path)
568         was_reingest = db_instance.get_video(video_id) is not None
569
570         # Validate (with-context variant surfaces ffprobe_meta for the report)
571         print("[CLI] Validating...")
572         val_results, val_context = registry.run_all_with_context(args.video_path,
global_config)
573         failures = [r for r in val_results if not r.passed]
574
575         # Save video status
576         status = "failed" if failures else "processed"
577         db_instance.add_video(video_id, args.video_path, status, [r.model_dump() for r
in val_results])
578
579         seg_name = args.segmenter or getattr(getattr(global_config, "indexing", None),
"default_segmenter", "motion")
580         segmenter_actual = None
581         segmentation_ran = False
582         try:
583             print("\n" + "="*40)
584             print("VALIDATION SUMMARY")
585             print("="*40)
586             for r in val_results:
587                 status_symbol = "[PASS]" if r.passed else "[FAIL]"
588                 print(f"{status_symbol} {r.validator_name}: {r.message}")
589             print("="*40 + "\n")
590
591             if failures:
592                 print("[FAIL] Video failed checks. Indexing halted.")
593                 return
594
595             # Index
596             segmenter_cls = segmenter_registry.get(seg_name)
597             if not segmenter_cls:
598                 print(f"[FAIL] Segmenter '{seg_name}' not found.")
599                 return
600
601             print(f"[CLI] Indexing play segments using segmenter: {seg_name}...")
602             segmenter_actual = seg_name
603             play_wm, cand_wm = db_instance.segment_watermarks(video_id)
604             segmenter = segmenter_cls(db_instance, global_config)
605             result = segmenter.process_video(video_id, args.video_path,
max_frames=args.max_frames)
606             segmentation_ran = True
607
608             if isinstance(result, Failure):
609                 print(f"[FAIL] Segmenter '{seg_name}' failed: {result.message}")
610                 print("[FAIL] Indexing halted for video.")
611                 segments = []

```

```

612         else:
613             segments = result.value if hasattr(result, "value") else result
614             print(f"[SUCCESS] Process complete. Indexed {len(segments)} play
segments inside database: {db_path}")
615             # Destroy-AFTER-confirm: keep new on success, restore prior on
empty/failure.
616             _finalize_reingest(video_id, segments, play_wm, cand_wm)
617         finally:
618             # Always emit the per-video observability report. Never raises.
619             paths = emit_indexer_report(
620                 db=db_instance, video_id=video_id, video_path=args.video_path,
config=global_config,
621                 val_results=val_results, val_context=val_context,
622                 segmenter_requested=args.segmenter, segmenter_actual=segmenter_actual,
623                 was_reingest=was_reingest, segmentation_ran=segmentation_ran,
624             )
625             if paths:
626                 print(f"[REPORT] Technical report : {paths.get('technical_md')}")
627                 print(f"[REPORT] Customer summary : {paths.get('customer_md')}")
628                 print(f"[REPORT] Machine JSON      : {paths.get('json')}")
629             return
630
631     # Start Local Server Mode
632     if args.server or not args.video_path:
633         print(f"[SERVER] Starting local FastAPI server on
http://localhost:{args.port}...")
634         # Resume self-scheduled work orphaned by the restart (EXECUTION_POLICY.md P3):
the
635         # in-process wake-timer for any parked 'waiting' job died with the old process,
so
636         # kick one drain ? it reclaims due jobs and re-arms the wake-timer for the rest.
637         _get_executor().submit(_drain_due_jobs)
638         uvicorn.run(app, host="127.0.0.1", port=args.port)
639
640 if __name__ == "__main__":
641     main()
642

```

4. user (2026-06-20T08:00:30.602Z)

```

1     """Chunked upload (B2, PR-2) ? extracted from backend.main into its own router.
2
3     Browser uploads arrive in sequential parts ? security.max_part_mb (free-tier edge
4     proxies cap request bodies at ~100 MB ? HOSTING_PLAN $2), are assembled server-side
5     under security.upload_dir, then submitted to /api/process by the returned path.
6     Upload state is in-process by design (this is the single-box alpha): a server restart
7     aborts partial uploads and the client re-inits. Per-user partitioning of upload_dir
8     arrives with PR-3 identity scoping.
9
10    The startup config lives on ``backend.main.global_config`` (set ONLY in ``main()``).
11    ``_upload_cfg`` resolves it through the ``backend.main`` module object so it always
12    reads the live value the server set at startup (and so tests that monkeypatch
13    ``backend.main.global_config`` flow through unchanged).
14    """
15    import os
16    import threading
17    import uuid
18    from typing import Any, Dict, Optional
19
20    from fastapi import APIRouter, HTTPException, Request
21    from pydantic import BaseModel
22
23    from backend.utils.hashing import compute_video_id # canonical file-identity hashing
24    from backend.utils.paths import safe_output_filename
25

```

```

26     router = APIRouter()
27
28     _uploads: Dict[str, Dict[str, Any]] = {}
29     _uploads_lock = threading.Lock()
30
31
32     class UploadInitRequest(BaseModel):
33         filename: str
34         total_size_bytes: Optional[int] = None
35
36
37     class UploadCompleteRequest(BaseModel):
38         total_parts: int
39
40
41     def _upload_cfg():
42         # Resolved lazily (import inside the function) to read the LIVE startup config that
43         # main() sets on the module global, and to avoid a circular import at module load
44         # (main.py imports this router; this module must not import main at import time).
45         import backend.main as _main
46         sec = getattr(_main.global_config, "security", None)
47         upload_dir = getattr(sec, "upload_dir", None) or "output/uploads"
48         max_bytes = int(getattr(sec, "max_upload_mb", 4096) or 4096) * 1024 * 1024
49         part_bytes = int(getattr(sec, "max_part_mb", 95) or 95) * 1024 * 1024
50         exts = [e.lower().lstrip(".") for e in (getattr(sec, "allowed_upload_extensions",
51 None) or ["mp4", "mov"])]
52         return upload_dir, max_bytes, part_bytes, exts
53
54     def _abort_upload(upload_id: str) -> None:
55         with _uploads_lock:
56             st = _uploads.pop(upload_id, None)
57             if st:
58                 try:
59                     os.remove(st["tmp_path"])
60                 except OSError:
61                     pass
62
63
64     @router.post("/api/upload/init")
65     def upload_init(req: UploadInitRequest):
66         """Start a chunked upload. Returns upload_id; PUT sequential parts next."""
67         upload_dir, max_bytes, part_bytes, exts = _upload_cfg()
68         try:
69             name = safe_output_filename(req.filename)
70         except ValueError as e:
71             raise HTTPException(status_code=400, detail=str(e)) from e
72         ext = os.path.splitext(name)[1].lower().lstrip(".")
73         if ext not in exts:
74             raise HTTPException(status_code=400,
75 detail=f"Extension '{ext}' not allowed. Allowed:
76 {sorted(exts)}")
77         if req.total_size_bytes is not None and req.total_size_bytes > max_bytes:
78             raise HTTPException(status_code=413,
79 detail=f"File exceeds the {max_bytes // (1024 * 1024)} MB
80 upload limit.")
81         os.makedirs(upload_dir, exist_ok=True)
82         upload_id = uuid.uuid4().hex
83         tmp_path = os.path.join(upload_dir, f".partial-{upload_id}")
84         open(tmp_path, "wb").close()
85         with _uploads_lock:
86             _uploads[upload_id] = {"filename": name, "tmp_path": tmp_path, "parts": 0,
87 "bytes": 0}
88         return {"upload_id": upload_id, "max_part_mb": part_bytes // (1024 * 1024),
89 "next_part": f"/api/upload/{upload_id}/part/0"}

```

```

87
88
89 @router.put("/api/upload/{upload_id}/part/{index}")
90 async def upload_part(upload_id: str, index: int, request: Request):
91     """Append one part (raw bytes body). Parts are strictly sequential (0,1,2,...)."""
92     _, max_bytes, part_bytes, _ = _upload_cfg()
93     with _uploads_lock:
94         st = _uploads.get(upload_id)
95         if st is None:
96             raise HTTPException(status_code=404, detail=f"Unknown upload_id '{upload_id}'")
97         if index != st["parts"]:
98             raise HTTPException(status_code=409,
99                                 detail=f"Out-of-order part: expected {st['parts']}, got
{index} (parts are sequential)")
100
101         received = 0
102         overflow = None
103         with open(st["tmp_path"], "ab") as fh:
104             async for chunk in request.stream():
105                 received += len(chunk)
106                 if received > part_bytes:
107                     overflow = f"Part exceeds the {part_bytes // (1024 * 1024)} MB per-part
limit."
108                     break
109                 if st["bytes"] + received > max_bytes:
110                     overflow = f"Upload exceeds the {max_bytes // (1024 * 1024)} MB total
limit."
111                     break
112                 fh.write(chunk)
113         if overflow:
114             _abort_upload(upload_id)
115             raise HTTPException(status_code=413, detail=overflow + " Upload aborted ? re-
init to retry.")
116
117         with _uploads_lock:
118             st["parts"] += 1
119             st["bytes"] += received
120         return {"upload_id": upload_id, "received_parts": st["parts"], "received_bytes":
st["bytes"]}
121
122
123 @router.post("/api/upload/{upload_id}/complete")
124 def upload_complete(upload_id: str, req: UploadCompleteRequest):
125     """Finalize: verify part count, move into place, return {path, video_id} for
/api/process."""
126     upload_dir, _, _, _ = _upload_cfg()
127     with _uploads_lock:
128         st = _uploads.get(upload_id)
129         if st is None:
130             raise HTTPException(status_code=404, detail=f"Unknown upload_id '{upload_id}'")
131         if st["parts"] == 0 or st["parts"] != req.total_parts:
132             raise HTTPException(status_code=409,
133                                 detail=f"Expected {req.total_parts} part(s), received
{st['parts']}.")
134         final_path = os.path.join(upload_dir, st["filename"])
135         if os.path.exists(final_path): # never overwrite an earlier upload with the same
name
136             stem, ext = os.path.splitext(st["filename"])
137             final_path = os.path.join(upload_dir, f"{stem}-{upload_id[:8]}{ext}")
138         os.replace(st["tmp_path"], final_path)
139         with _uploads_lock:
140             _uploads.pop(upload_id, None)
141         video_id = compute_video_id(final_path)
142         return {"path": os.path.abspath(final_path), "video_id": video_id,
143                 "size_bytes": os.path.getsize(final_path),

```

```

144         "next": "POST /api/process with this path"}
145
146
147 @router.delete("/api/upload/{upload_id}")
148 def upload_abort(upload_id: str):
149     """Abort an in-flight upload and delete its partial file."""
150     with _uploads_lock:
151         known = upload_id in _uploads
152         if not known:
153             raise HTTPException(status_code=404, detail=f"Unknown upload_id '{upload_id}'")
154         _abort_upload(upload_id)
155     return {"status": "aborted", "upload_id": upload_id}
156

```

5. user (2026-06-20T08:00:30.918Z)

```

backend\api\__init__.py
backend\api\uploads.py
backend\api\job_worker.py

```

6. assistant (2026-06-20T08:00:34.084Z)

Now let me read the `job_worker` router and check for any other routers or static frontend JS that calls the API. Let me also read the hosting/auth docs.

7. user (2026-06-20T08:00:35.047Z)

```

1     """Async job worker ? drain/wake loop (DEFERRED_HARDENING_PLAN #16, EXECUTION_POLICY.md
P3).
2
3     Extracted verbatim from backend.main to shrink the orchestrator; behavior is unchanged.
4     The worker DRAINS the jobs table via `claim_due_job` and honours WAIT_AND_RESUME: a job
5     that raises `JobWaiting` is parked (status 'waiting' + `next_poll_at` + progress) and
6     re-claimed when due ? with no central scheduler (the DB is the clock).
7
8     The monkeypatch contract these functions rely on (and that tests depend on):
9     every cross-function seam ? `db_instance`, `global_config`, `_execute_processing`,
10    `_arm_wake_timer`, `_get_executor`, `_MAX_DRAIN_WAKE_SEC` ? is resolved THROUGH the
11    `backend.main` module object at call-time (`import backend.main as _main`), never as a
12    bare local. `backend.main` re-exports these names, so `monkeypatch.setattr(backend.main,
13    ...)`` still intercepts the inter-function calls (e.g. a parked job arms the *patched*
14    `_arm_wake_timer`, so no real daemon timer leaks in tests). Importing `backend.main`
lazily
15    inside the functions also avoids the circular import at module load (main.py imports
this
16    module; this module must not import main at import time).
17    """
18    import logging
19    import threading
20    import traceback
21    from concurrent.futures import ThreadPoolExecutor
22    from typing import Any, Dict, Optional
23
24    logger = logging.getLogger(__name__)
25
26    # Async job executor (DEFERRED_HARDENING_PLAN #16). max_workers=1 on purpose: a single
27    # self-hosted box serializes GPU work, and the Gemini provider is rate-fragile under
28    # parallel uploads (2026-06-10 ops learning: serial uploads + backoff). The segmenters
29    # already parallelize their AI handoff internally via their own pools.
30    _job_executor: Optional[ThreadPoolExecutor] = None
31
32
33    def _get_executor() -> ThreadPoolExecutor:
34        """Lazy module-global executor; tests monkeypatch this for inline execution."""

```

```

35     global _job_executor
36     if _job_executor is None:
37         _job_executor = ThreadPoolExecutor(max_workers=1,
thread_name_prefix="jobworker")
38     return _job_executor
39
40
41     class JobWaiting(Exception):
42         """Raised inside the pipeline to SELF-SCHEDULE a job (EXECUTION_POLICY.md P3,
43         WAIT_AND_RESUME). It means "I can't finish now ? park me and resume in `delay_sec`,
44         NOT a failure. The worker persists `next_poll_at` + `progress` (via
`set_job_waiting`),
45         releases the thread, and re-claims the job when due (`claim_due_job`). No work to
date
46         is the segmenter's `_execute_processing`, run UNDER the destroy-after-confirm
contract.
47
48         The wiring is additive: the production segmenter path never raises this today (zero
behaviour change), so a job runs exactly as before. The hook is what a later slice
50         (resumable AI handoff) raises once a long op exhausts its in-process wait budget.
51         """
52
53         def __init__(self, delay_sec: float, progress: Optional[str] = None) -> None:
54             super().__init__(f"job parked for {delay_sec:.0f}s")
55             self.delay_sec = max(0.0, float(delay_sec))
56             self.progress = progress
57
58
59     def _job_to_request(job: Dict[str, Any]):
60         """Reconstruct the submit-time ProcessRequest from a claimed job row, so a job
61         re-claimed by `claim_due_job` (queued OR resumed-from-waiting) runs identically to
62         the original submit. The row stores exactly the ProcessRequest fields."""
63         import backend.main as _main
64         return _main.ProcessRequest(
65             video_path=job["video_path"],
66             player_pool=job.get("player_pool"),
67             max_frames=job.get("max_frames"),
68             segmenter=job.get("segmenter"),
69         )
70
71
72     def _run_claimed_job(job: Dict[str, Any]) -> None:
73         """Run ONE job already claimed (`status='running'`) by `claim_due_job`, recording
its
74         terminal state ? OR parking it for resume on `JobWaiting` (WAIT_AND_RESUME).
75
76         Job `status` is the EXECUTION state (queued|waiting|running|done|failed): 'done'
means
77         the worker finished ? the pipeline's own verdict (validation failure etc.) lives in
78         result["status"]; 'failed' means the worker itself crashed/raised; 'waiting' means
the
79         job self-scheduled and will be re-claimed when `next_poll_at` is due.
80         """
81         import backend.main as _main
82         db_instance = _main.db_instance
83         job_id = job["id"]
84         req = _main._job_to_request(job)
85         try:
86             result = _main._execute_processing(
87                 req, progress=lambda stage: db_instance.set_job_progress(job_id, stage))
88             if result.get("video_id"):
89                 db_instance.set_job_video_id(job_id, result["video_id"])
90                 db_instance.mark_job_done(job_id, result)
91         except _main.JobWaiting as w:
92             # Cooperative re-enqueue: persist progress + next_poll_at, release the worker.

```



```

93         logger.info(f"Job {job_id} parked for {w.delay_sec:.0f}s (resume when due).")
94         db_instance.set_job_waiting(job_id, w.delay_sec, progress=w.progress)
95     except Exception as e:
96         logger.error(f"Job {job_id} crashed: {e}\n{traceback.format_exc()}")
97         db_instance.mark_job_failed(job_id, f"{type(e).__name__}: {e}")
98
99
100     def _drain_due_jobs() -> int:
101         """One worker tick: claim and run every currently-runnable job (queued, or waiting
whose
102         `next_poll_at` is due), earliest-due first, until the table has nothing runnable.
103
104         Each parked job (`JobWaiting` ? status 'waiting') drops out of this drain and is
picked
105         up by a FUTURE drain once due; we schedule that follow-up drain via
`_schedule_drain` so
106         a single-worker box keeps making progress with no central timer (the DB is the
clock).
107         Returns the number of jobs that reached a terminal/parked state this tick.
108         """
109         import backend.main as _main
110         ran = 0
111         while True:
112             job = _main.db_instance.claim_due_job()
113             if job is None:
114                 break
115             ran += 1
116             _main._run_claimed_job(job)
117         # If anything is parked-but-not-yet-due, arrange a wake-up so it resumes without a
new
118         # submit. On the single-box executor this is a delayed re-drain; multi-worker just
relies
119         # on the next submit/drain. Best-effort ? never raises into the worker.
120         _main._schedule_next_drain_if_waiting()
121         return ran
122
123
124     def _schedule_next_drain_if_waiting() -> None:
125         """If any job is parked in 'waiting', arm a re-drain for when the soonest one is
due, so a
126         self-scheduled job resumes with no further client submit. The drain itself re-checks
127         due-ness, so an early wake is harmless and a clamped wake for a far-future job is
fine."""
128         import backend.main as _main
129         try:
130             delay = _main.db_instance.seconds_until_next_due()
131         except Exception as e: # pragma: no cover - defensive; never wedge the worker
132             logger.error(f"Could not compute next-due delay: {e}")
133             return
134         if delay is None:
135             return # nothing waiting
136         # Clamp so a far-future job doesn't pin a thread forever; the drain re-checks due-
ness.
137         _main._arm_wake_timer(max(0.0, min(delay, _main._MAX_DRAIN_WAKE_SEC)))
138
139
140     def _arm_wake_timer(delay_sec: float) -> None:
141         """Arm a single daemon timer that re-submits a drain after ``delay_sec`` (no central
142         scheduler ? the DB is the clock). Isolated as the sole timer-creating seam so tests
can
143         monkeypatch it to a no-op and assert the *scheduling decision* without real
waiting."""
144         import backend.main as _main
145         timer = threading.Timer(delay_sec, lambda:
_main._get_executor().submit(_main._drain_due_jobs))

```

```

146         timer.daemon = True
147         timer.start()
148
149
150     #: A parked job further out than this is re-checked by a capped wake rather than one
long
151     #: sleeping timer ? keeps the wake cadence bounded without a central scheduler.
152     _MAX_DRAIN_WAKE_SEC = 60.0
153

```

8. user (2026-06-20T08:00:35.521Z)

```

1     """HTTP API surface for the indexer (FastAPI routers)."""
2

```

9. user (2026-06-20T08:00:37.369Z)

```

backend\api\uploads.py:64:@router.post("/api/upload/init")
backend\api\uploads.py:89:@router.put("/api/upload/{upload_id}/part/{index}")
backend\api\uploads.py:123:@router.post("/api/upload/{upload_id}/complete")
backend\api\uploads.py:147:@router.delete("/api/upload/{upload_id}")
backend\main.py:52:@app.get("/", response_class=HTMLResponse)
backend\main.py:121:@app.get("/api/config")
backend\main.py:240:@app.post("/api/process", status_code=202)
backend\main.py:274:@app.get("/api/jobs/{job_id}")
backend\main.py:307:@app.get("/api/highlights/{video_id}/{filename}")
backend\main.py:331:@app.post("/api/query")
backend\main.py:342:@app.post("/api/compile")

```

10. assistant (2026-06-20T08:00:41.091Z)

I have the full endpoint list. Now I confirmed there are no other routers (only `uploads\_api.router` is included). Let me check for any auth/middleware/dependency wiring and look at the `query\_play\_segments` DB shape, plus read the hosting/auth docs.

11. user (2026-06-20T08:00:42.991Z)

```

backend\annotations\automated_provider.py:5:between authoritative human-vendor runs (the slow
tier).
backend\main.py:39:# auth/tenancy work; this tool uses no cookies, so credentials stay off.
backend\main.py:249:     allowed_root = getattr(getattr(global_config, "security", None),
"allowed_video_root", None)
backend\eval\ablation_pdf.py:138:                                     author="badminton-highlight-
indexer")
backend\api\uploads.py:3:Browser uploads arrive in sequential parts ? security.max_part_mb
(free-tier edge
backend\api\uploads.py:5:under security.upload_dir, then submitted to /api/process by the
returned path.
backend\api\uploads.py:46:     sec = getattr(_main.global_config, "security", None)
backend\infrastructure\database.py:221:         # `user_id` to videos + candidate_segments so a
future per-user / multi-tenant
backend\infrastructure\database.py:224:         # filters on user_id) is unaffected. ALTER (not
recreate) so v1-v3 rows survive;
backend\infrastructure\database.py:227:         self._add_column_idempotent(cursor, "ALTER TABLE
videos ADD COLUMN user_id TEXT")
backend\infrastructure\database.py:228:         self._add_column_idempotent(cursor, "ALTER TABLE
candidate_segments ADD COLUMN user_id TEXT")
backend\infrastructure\database.py:229:         # Partial indexes (WHERE user_id IS NOT NULL)
keep the NULL=local fast path free
backend\infrastructure\database.py:232:         "CREATE INDEX IF NOT EXISTS idx_videos_user
ON videos(user_id) "
backend\infrastructure\database.py:233:         "WHERE user_id IS NOT NULL")
backend\infrastructure\database.py:236:         "ON candidate_segments(user_id) WHERE user_id
IS NOT NULL")

```

```

backend\infrastructure\database.py:240:         # v5: per-user model store
(PERSONALIZATION_PLAN.md S2). One row per (user_id,
backend\infrastructure\database.py:242:         # `classifier_json` (tenant-safe ? no file
artifact), plus the promotion evidence
backend\infrastructure\database.py:247:         #   user_id           ? owner of the model
(NULL allowed = the local install)
backend\infrastructure\database.py:259:         user_id TEXT,
backend\infrastructure\database.py:276:         ON user_models(user_id, version)
backend\infrastructure\database.py:279:         # (SQLite treats NULL user_id rows as distinct,
so the local install's single
backend\infrastructure\database.py:283:         ON user_models(user_id) WHERE is_active =
1
backend\infrastructure\database.py:633:         # (`fusion_hybrid._select_for_handoff`) is the NEXT,
owner-gated PR. NULL user_id =
backend\infrastructure\database.py:637:         def upsert_user_model(self, user_id: Optional[str],
*, version: int = 1,
backend\infrastructure\database.py:645:         """Insert or replace ONE per-user model row keyed
by `(user_id, version)``.
backend\infrastructure\database.py:657:         # Demote the current active row first
(NULL user_id = local: IS NULL match).
backend\infrastructure\database.py:658:         if user_id is None:
backend\infrastructure\database.py:661:             "WHERE user_id IS NULL AND
is_active = 1 AND version != ?",
backend\infrastructure\database.py:666:             "WHERE user_id = ? AND
is_active = 1 AND version != ?",
backend\infrastructure\database.py:667:             (user_id, version))
backend\infrastructure\database.py:670:             (user_id, version,
baseline_version, feature_schema_hash,
backend\infrastructure\database.py:674:             ON CONFLICT(user_id, version) DO
UPDATE SET
backend\infrastructure\database.py:682:             (user_id, version, baseline_version,
feature_schema_hash,
backend\infrastructure\database.py:690:             + ("user_id IS NULL" if user_id is
None else "user_id = ?")
backend\infrastructure\database.py:692:             ((version,) if user_id is None else
(user_id, version))).fetchone()
backend\infrastructure\database.py:695:             logger.error(f"Failed to upsert user_model
(user={user_id}, v={version}): {e}")
backend\infrastructure\database.py:698:         def get_active_user_model(self, user_id:
Optional[str]) -> Optional[Dict[str, Any]]:
backend\infrastructure\database.py:699:         """Return the single active per-user model row
for `user_id` (NULL = local install),
backend\infrastructure\database.py:703:             if user_id is None:
backend\infrastructure\database.py:705:                 "SELECT * FROM user_models WHERE
user_id IS NULL AND is_active = 1").fetchone()
backend\infrastructure\database.py:708:                 "SELECT * FROM user_models WHERE
user_id = ? AND is_active = 1",
backend\infrastructure\database.py:709:                 (user_id,)).fetchone()
backend\config\models.py:258:         # google.auth, grpc) emit one INFO line per request ? dozens
per Gemini run, which
backend\config\models.py:264: class SecurityConfig(BaseModel):
backend\config\models.py:265:         # When set, /api/process video_path must resolve under this
directory (hosted/tenant
backend\config\models.py:298:         security: SecurityConfig =
Field(default_factory=SecurityConfig)
backend\eval\classifier.py:4: no heavy ML stack ? the project already depends on numpy, and
sklearn is not
backend\reporting\harvest.py:145:         key_env = f"{provider.upper()}_API_KEY"
backend\reporting\harvest.py:153:         "api_key_present": bool(os.environ.get(key_env)) if
ai_based else None,
backend\providers\__init__.py:14:         def get(self, name: str, api_key: str, model_name: str) ->
AIVideoProvider:
backend\providers\__init__.py:17:         return self._registry[name](api_key=api_key,
model_name=model_name)
backend\utils\paths.py:42:         allowed_root (a future hosted/tenant deployment) enforces

```

```

containment.
backend\providers\gemini.py:20: def __init__(self, api_key: str, model_name: str):
backend\providers\gemini.py:21:     super().__init__(api_key, model_name)
backend\providers\gemini.py:22:     # Initialize client lazily to avoid connection/auth
issues during class instantiation
backend\providers\gemini.py:28:         self._client = genai.Client(api_key=self.api_key)
backend\utils\logging_setup.py:5: Gemini path (`google-genai` ->
`httpx`/`grpc`/`google.auth`) that is dozens of
backend\utils\logging_setup.py:26:     "google.auth",
backend\providers\base.py:5: def __init__(self, api_key: str, model_name: str):
backend\providers\base.py:6:     self.api_key = api_key

```

[Showing results with pagination = limit: 60]

12. user (2026-06-20T08:00:43.507Z)

Found 3 files
 backend\main.py
 backend\infrastructure\database.py
 tests\test_reporting_harvest.py

13. user (2026-06-20T08:00:44.057Z)

docs\HOSTING_PLAN.md

14. user (2026-06-20T08:00:44.443Z)

docs\UI_ALPHA_EXECUTION_PLAN.md

15. assistant (2026-06-20T08:00:48.499Z)

No auth dependencies anywhere in the API layer ? confirmed (no `Depends`, no `HTTPBearer`, no token check). The `user\_id` columns exist in the DB schema but are not wired to any endpoint. Let me confirm the `query\_play\_segments` response shape and read the two docs for the intended auth model.

16. user (2026-06-20T08:00:50.394Z)

```

300         "filepath=excluded.filepath, status=excluded.status, "
301         "validation_results=excluded.validation_results",
302         (video_id, filepath, status, json.dumps(validation_results)),
303         "Failed to add video",
304     )
305
306     def segment_watermarks(self, video_id: str) -> Tuple[int, int]:
307         """Return (max play_segment id, max candidate_segment id) for a video (0 if
308         none).
309         Captured before a (re)segmentation so the run's new rows (id > watermark) can be
310         told apart from prior rows (id <= watermark), enabling destroy-AFTER-confirm.
311         """
312         with self._connect() as conn:
313             cur = conn.cursor()
314             p = cur.execute("SELECT COALESCE(MAX(id), 0) FROM play_segments WHERE
315             video_id = ?",
316                             (video_id,)).fetchone()[0]
317             c = cur.execute("SELECT COALESCE(MAX(id), 0) FROM candidate_segments WHERE
318             video_id = ?",
319                             (video_id,)).fetchone()[0]
320             return int(p), int(c)
321
322     def prune_segments(self, video_id: str, *, play_upto: Optional[int] = None,
323                       play_after: Optional[int] = None, cand_upto: Optional[int] =
324                       None,

```

```

322             cand_after: Optional[int] = None) -> None:
323         """Delete play/candidate segments by id range (events cascade from
play_segments).
324
325         Used to finish a (re)segmentation: keep the NEW rows and drop the OLD ones on a
326         successful run (`*_upto` = the pre-run watermark), or drop the NEW partial
rows
327         and keep the OLD ones when the run failed/produced nothing (`*_after`).
328         """
329         with self._connect() as conn:
330             cur = conn.cursor()
331             if play_upto is not None:
332                 cur.execute("DELETE FROM play_segments WHERE video_id = ? AND id <= ?",
(video_id, play_upto))
333             if play_after is not None:
334                 cur.execute("DELETE FROM play_segments WHERE video_id = ? AND id > ?",
(video_id, play_after))
335             if cand_upto is not None:
336                 cur.execute("DELETE FROM candidate_segments WHERE video_id = ? AND id <=
?", (video_id, cand_upto))
337             if cand_after is not None:
338                 cur.execute("DELETE FROM candidate_segments WHERE video_id = ? AND id >
?", (video_id, cand_after))
339             conn.commit()
340
341         def add_play_segment(self, video_id: str, start_time: float, end_time: float,
server: Optional[str] = None, receiver: Optional[str] = None, metadata: Optional[Dict[str, Any]]
= None) -> Optional[int]:
342             try:
343                 with self._connect() as conn:
344                     cursor = conn.cursor()
345                     cursor.execute(
346                         "INSERT INTO play_segments (video_id, start_time, end_time,
duration, server, receiver, metadata) VALUES (?, ?, ?, ?, ?, ?, ?)",
347                         (video_id, start_time, end_time, end_time - start_time, server,
receiver, json.dumps(metadata or {})))
348                     )
349                     conn.commit()
350                     return cursor.lastrowid
351             except Exception as e:
352                 logger.error(f"Failed to add play segment: {e}")
353                 return None
354
355         def add_event(self, segment_id: int, timestamp: float, event_type: str, player:
Optional[str] = None, details: Optional[Dict[str, Any]] = None) -> bool:
356             return self._execute_write(
357                 "INSERT INTO events (segment_id, timestamp, type, player, details) VALUES
(?, ?, ?, ?, ?)",
358                 (segment_id, timestamp, event_type, player, json.dumps(details or {})),
359                 "Failed to add event",
360             )
361
362         def add_candidate_segment(self, video_id: str, detector: str, start_time: float,
end_time: float,
363                                 chunk_index: Optional[int] = None, confidence:
Optional[float] = None,
364                                 stats: Optional[Dict[str, Any]] = None,
365                                 detection_params: Optional[Dict[str, Any]] = None) ->
Optional[int]:
366             """Persist one raw (pre-confirmation) detection. `stats`/`detection_params` are
367             detector-specific dicts stored as JSON. Returns the new candidate id, or
None."""
368             try:
369                 with self._connect() as conn:
370                     cursor = conn.cursor()

```

```

371         cursor.execute(
372             """INSERT INTO candidate_segments
373                 (video_id, detector, chunk_index, start_time, end_time, duration,
374                 confidence, stats, detection_params)
375                 VALUES (?, ?, ?, ?, ?, ?, ?, ?, ?)""",
376             (video_id, detector, chunk_index, start_time, end_time, end_time -
start_time,
377             confidence, json.dumps(stats or {}), json.dumps(detection_params or
{})))
378         )
379         conn.commit()
380         return cursor.lastrowid
381     except Exception as e:
382         logger.error(f"Failed to add candidate segment: {e}")
383         return None
384
385     def link_candidate_to_segment(self, candidate_id: int, segment_id: int) -> bool:
386         """Record that a candidate detection was confirmed as a given play_segment."""
387         return self._execute_write(
388             "UPDATE candidate_segments SET confirmed_segment_id = ? WHERE id = ?",
389             (segment_id, candidate_id),
390             f"Failed to link candidate {candidate_id} to segment {segment_id}",
391         )
392
393     def query_candidate_segments(self, video_id: str, detector: Optional[str] = None,
394                                 only_unlabeled: bool = False) -> List[Dict[str, Any]]:
395         """Fetch candidate detections for the annotation / fine-tuning workflow.
396         `stats` and `detection_params` are deserialized from JSON."""
397         query = "SELECT * FROM candidate_segments WHERE video_id = ?"
398         params: List[Any] = [video_id]
399
400         if detector:
401             query += " AND detector = ?"
402             params.append(detector)
403         if only_unlabeled:
404             query += " AND human_label IS NULL"
405
406         query += " ORDER BY start_time"
407
408         candidates = []
409         with self._connect() as conn:
410             rows = conn.cursor().execute(query, params).fetchall()
411             for row in rows:
412                 d = dict(row)
413                 d["stats"] = json.loads(d["stats"] or "{}")
414                 d["detection_params"] = json.loads(d["detection_params"] or "{}")
415                 candidates.append(d)
416         return candidates
417
418     def get_video(self, video_id: str) -> Optional[Dict[str, Any]]:
419         with self._connect() as conn:

```

17. user (2026-06-20T08:00:50.540Z)

```

1     # Hosting Plan ? khelsutra.guru
2
3     **Status:** PROPOSAL ? awaiting owner ratification.
4     **Date:** 2026-06-10 · **Constraints:** budget $10?25/mo · GCP credits available ·
DigitalOcean
5     acceptable · alpha backend = owner's GPU box behind a tunnel · 10?25 alpha testers ·
registrar = GoDaddy.
6     **Companion:** `docs/UI_PROPOSAL.md`.
7
8     ## 1. Recommendation in one line
9

```

```

10      **Alpha: free edge + your box** (Cloudflare DNS/Pages/Access/Tunnel ? FastAPI on the GPU
box) at
11      **$0/mo**, saving GCP credits; **Beta: GCP control plane + your box as pull-worker**
(Cloud Run +
12      GCS + Firebase Auth, credits-covered), keeping the edge where it's free.
13
14      Rationale for the Cloudflare pieces despite the GCP preference: GCP has no free
equivalent of the
15      Tunnel + Access combo (IAP requires a load balancer, ~$18/mo standing cost), and the
alpha needs
16      exactly three things GCP charges for but Cloudflare gives free: a secure tunnel to a
home box with
17      a custom domain, email-allowlist auth (free ?50 users ? fits 10?25 testers), and static
hosting.
18      Credits are better spent at beta on compute/storage, which is where GCP is excellent.
19
20      ## 2. Alpha topology ($0/mo)
21
22      ```
23      tester browser
24      ? https
25      ?
26      Cloudflare edge (free): DNS + TLS + Access (email-OTP allowlist)
27      ??? app.khelsutra.guru ? Cloudflare Pages (React SPA; auto-deploy from GitHub)
28      ??? api.khelsutra.guru ? Cloudflare Tunnel (outbound-only `cloudflared` service on
the box)
29
30      ?
31      ?
32      FastAPI :8000 on the GPU box (Windows)
33      ? Gemini segmenter path (no GPU in request path)
34      ? async jobs (#16), SQLite WAL, output/ on disk
35
36      - **No inbound ports opened** on the home network; `cloudflared` dials out. Home IP
never exposed.
37      - **Auth:** Cloudflare Access in front of BOTH hostnames; backend trusts the
38      `Cf-Access-Authenticated-User-Email` header (verify the Access JWT for rigor) for per-
user scoping.
39      - **Upload limit:** free-tier proxy caps request bodies at **100 MB** ? SPA uploads in
?90 MB
40      chunks (UI_PROPOSAL B2). Match videos (1?4 GB) flow fine as chunks.
41      - **Box offline = service offline.** Acceptable for alpha; SPA shows a friendly
"processing engine
42      offline" banner (a `/api/health` ping). Beta removes this with the queue.
43      - **Costs:** Pages, Tunnel, Access, DNS = $0. Only the Gemini per-run cost
(~$0.05/match) and
44      domain renewal.
45
46      ## 3. Beta topology (GCP credits; ~$15?20/mo post-credits)
47
48      ```
49      browser ??? Cloudflare edge (free, unchanged) ??? app.khelsutra.guru (Pages)
50      ??? api.khelsutra.guru ??? Cloud Run
(FastAPI control plane, scale-to-zero)
51
52      ? jobs
table (Cloud SQL Postgres)
53
54      ? signed
resumable URLs
55
56      ?
?
GCS buckets
(uploads / reels, lifecycle ? cold)
57
58      ? pull
jobs, outbound-only
59
60      GPU box = first
`byo_worker` (PLATFORM P3)

```

```

57     ``
58
59     - Uploads go **browser ? GCS directly** (resumable signed URLs) ? no proxy body limit,
no home
60     bandwidth for ingest; the box pulls only what it processes (WASB/owned-model jobs),
Cloud Run
61     handles Gemini-path jobs entirely in-cloud.
62     - Auth graduates to **Firebase Auth** (free; Google + email sign-in; India-friendly
phone OTP
63     later); Access allowlist retires.
64     - Post-credits estimate: Cloud Run ~$0?5 (free tier: 2 M req + 180 K vCPU-s/mo, scale-
to-zero) .
65     GCS 100 GB ? $2.60 + egress $0.12/GB (reel downloads ? watch this line) . Cloud SQL
smallest
66     ? $10 ? **~$15?20/mo**, inside budget. Egress lever if it grows: serve reels from
Cloudflare R2
67     (zero egress) while keeping everything else GCP.
68
69     ## 4. Alternatives considered
70
71     | Option | Monthly (alpha ? beta) | Pros | Cons |
72     |---|---|---|---|
73     | **Recommended hybrid** (CF edge + box ? +GCP) | **$0 ? ~$0 w/ credits** (~$15?20
after) | Cheapest at every phase; credits used where they matter; maps to PLATFORM P2/P3 | Two
vendors (CF + GCP) |
74     | **All-DigitalOcean** | ~$11 ? ~$17 ($6 droplet + Spaces $5: 250 GiB + **1 TiB egress
included**) | One simple vendor; Spaces' included egress is the best video-download deal; per-
second billing | No credits; no scale-to-zero; still needs a tunnel-equivalent to reach the box
(run `frp`/WireGuard on the droplet) |
75     | **All-GCP from day 1** | ~$18+ ? credits | Single vendor, credits immediately | LB+IAP
standing cost just to gate an alpha; no free tunnel to the home box; credits burn on plumbing |
76     | **All-Cloudflare** (Pages+Workers+R2) | ~$0 ? ~$5 | Zero egress on R2; cheapest
forever | FastAPI doesn't run on Workers (API stays on box/Cloud Run anyway); ignores GCP
credits |
77
78     ## 5. Domain setup ? GoDaddy ? Cloudflare DNS (one-time, ~15 min, you)
79
80     1. Create a free Cloudflare account ? **Add a domain** ? `khelsutra.guru` ? Free plan.
Cloudflare
81     imports existing records and shows **two nameservers** (e.g.
`xxx.ns.cloudflare.com`).
82     2. GoDaddy ? **My Products ? khelsutra.guru ? Manage DNS ? Nameservers ? Change ?
83     "I'll use my own nameservers"** ? paste the two Cloudflare nameservers ? Save.
Propagation:
84     minutes to 24 h. (Registration/renewal stays at GoDaddy; only DNS moves. Optional
later:
85     transfer the registration to Cloudflare Registrar for at-cost .guru renewals.)
86     3. Records (I manage after takeover): `app` ? Pages project; `api` ? `**Cloudflare API token** (you create: dash ? My Profile ? API Tokens) | Zone:DNS:Edit,
Account:Pages:Edit, Account:Access:Apps:Edit, Account:Tunnel:Edit ? scoped to this zone | DNS
records, Pages deploys (wrangler), Access policies, tunnel config |
94     | **GitHub repo** `khelsutra` (private) | Pages connects once (dashboard, you; build
root = `web/`) ? every push auto-deploys | I write the SPA in your Projects folder; commits
pushed = deployed |
95     | **On the box** (one-time, I hand you exact commands) | `cloudflared` installed as a
Windows service + `config.yml` | The tunnel itself |
96     | **GCP (beta only)** | A `khelsutra` project + service account (Cloud Run admin,
Storage admin, Cloud SQL admin) | Control-plane deploys |

```



```

97
98     Day-to-day flow after setup: I build ? commit ? push ? Pages deploys the SPA
automatically;
99     API/tunnel/DNS changes via the scoped token. Secrets (Gemini key, Access policy) never
live in the
100     repo; `.env` on the box + Cloudflare/GCP dashboards.
101
102     ## 7. Go-live security checklist (alpha)
103
104     Access allowlist ON for both hostnames ? CORS narrowed to `https://app.khelsutra.guru` ?
rate
105     limit + per-user upload quota ? `security.allowed_video_root` set ? upload size cap +
extension
106     allowlist ? **rotate the Gemini API key** (NEXT_STEPS housekeeping ? it was shared in
chat) ?
107     confirm no router port-forwards ? weekly backup: SQLite DB + `output/` ? GCS bucket
(rclone
108     scheduled task; credits).
109
110     ## 8. Decisions needed from owner
111
112     1. Ratify the **hybrid recommendation** (CF free edge now, GCP credits at beta)? Or
force
113         single-vendor (DO / GCP) despite the cost notes above?
114     2. OK to move **DNS to Cloudflare** (registration stays at GoDaddy)?
115     3. Beta reel storage: **GCS (credits)** vs **R2 (zero egress)** vs **DO Spaces (1 TiB
included)** ?
116         can defer until beta Drop 2.
117     4. Approve the access grants in $6 so I can run the setup end-to-end.
118
119     *Ratified choices get recorded in `docs/archives/decisions/DECISIONS.md`.*
120

```

18. assistant (2026-06-20T08:00:54.177Z)

The auth model is clear from the hosting plan. Let me read the UI Alpha execution plan to confirm the intended per-rally-selection UI requirements and any auth specifics, plus confirm the `query\_play\_segments` JSON shape and the `get\_video` shape.

19. user (2026-06-20T08:00:55.608Z)

```

1     # UI Alpha Execution Plan ? KhelSutra Guru
2
3     **Status:** ACTIVE · updated 2026-06-10 · **Linked:** `UI_PROPOSAL.md` ·
`HOSTING_PLAN.md` ·
4     `mockups/ui_alpha_mockup.html` · ADR-009 (`archives/decisions/DECISIONS.md`).
5     **Sequencing principle (owner-requested):** build and test **everything locally first**
6     deployment (DNS/tunnel/Pages/Access) starts only after the local end-to-end gate passes.
7
8     ## Decisions locked
9
10    | Date | Decision |
11    |---|---|
12    | 2026-06-10 | Product name **KhelSutra Guru** · domain **khelsutra.guru** (GoDaddy,
3-yr + protection) |
13    | 2026-06-10 | Separate **private repo `khelsutra`** ("heart and brain"): `web/` SPA
now, control plane at beta; backend enablers stay in the indexer repo |
14    | 2026-06-10 | **Local-first delivery** (this plan's phase structure) |
15    | open | Final "go" on PR-1 coding start (= #16 greenlight) · training opt-in default
OFF · invite list · stack (React+Vite+TS+Tailwind stands unless owner objects) |
16
17    ## Master checklist
18
19    ### G0 ? Setup (now)
20    - [x] Name/domain/repo-split ratified; spelling confirmed via GoDaddy receipt

```

```

21 - [x] `khelsutra` repo **scaffolded** at `C:\Users\avidu\Projects\khelsutra` (README,
.gitignore, web/, docs/)
22 - [x] **Owner:** git-init + private GitHub push ? done 2026-06-10:
23   `https://github.com/avidullu/khelsutra` (private; git runs Windows-side per ADR-009)
24 - [x] **Owner:** explicit "go" for PR-1 ? given 2026-06-10
25 - [x] **Owner:** Gemini API key rotated ? 2026-06-10, owner-verified (confirm old-key
deletion in AI Studio)
26
27   ### Phase L ? build & test locally (no cloud; ?6?7 sessions)
28
29   **L1 ? backend enablers** (indexer repo · branch off master · ONE PR each, no stacking ·
offline tests in every PR · GPU/real-video checks = owner):
30 - [x] **PR-1 · B1 async job model (#16):** `jobs` table (user_version 1?2), `POST
/api/process` ? 202 + `job_id`, `GET /api/jobs/{job_id}`, ThreadPoolExecutor worker,
restart/crash state handling ? **MERGED 2026-06-11, PR #87** (CI green: 333 passed; machine-side
real-video checks fold into L3)
31 - [x] **PR-2 · B2+B3 upload & download:** chunked upload (init ? sequential ?95 MB parts
? complete ? MD5; caps + mp4/mov allowlist; per-user dirs deferred to PR-3) + streaming `GET
/api/highlights/{video_id}/{name}` ? **MERGED 2026-06-11, PR #99** (also fixes static-UI
`segment_ids` compile bug + adds browser download link). **Live E2E verified at merge:** 86 MB
real video uploaded in 3 sequential parts and reassembled to MD5 byte-identity; out-of-order 409
/ traversal 400 (handler) + 404 (router) / unknown-id 404 confirmed; compile of a real confirmed
rally ? streamed ? download (3.9 MB reel).
32 - [ ] **PR-3 · B4+B7 identity & exposure:** `owner_email` column + scoping on every
query; Access-JWT/header read with `DEV_USER_EMAIL` env fallback for local dev; CORS narrowed;
rate limit; per-user quota
33 - [ ] **PR-4 · B6 correction loop:** `PATCH /api/segments/{id}` accept/reject/boundary-
nudge ? `candidate_segments.human_label`/`notes` + provenance (who/when); retire
`backend/api/static` (B5)
34
35   **L2 ? SPA** (khelsutra repo, `web/`; dev server proxies to `http://localhost:8000`):
36 - [ ] **M1 scaffold:** Vite + React + TS + Tailwind, router, typed API client, branding
(KhelSutra Guru)
37 - [ ] **M2:** A2 Upload (chunked, guidelines checklist) + A3 Job Progress (poll)
38 - [ ] **M3:** A1 Matches + A4 Rally Review (player seek, accept/reject/nudge wired to
PR-4)
39 - [ ] **M4:** A5 Highlights (compile + download) + A6 Report + engine-offline banner
40 - [ ] Unit tests for API client + review-state logic; `npm run build` clean
41
42   **L3 ? local end-to-end (owner's box, real match videos):**
43 - [ ] Full UI flow on localhost: upload ? job ? review/correct ? compile ? download
44 - [ ] `python run_all_tests.py` green; UI polling vs SQLite WAL sanity; WASB/CLI paths
untouched
45 - [ ] **GATE GL:** owner signs off local E2E ? only then deployment work begins
46
47   ### Phase D ? deployment (?2 sessions + ~1 hr owner)
48 - [ ] **D1 (owner):** Cloudflare account ? add khelsutra.guru ? GoDaddy nameserver
switch (HOSTING_PLAN $5) ? scoped API token ($6) ? Pages project connected to GitHub `khelsutra`
(build root `web/`)
49 - [ ] **D2 (Claude):** DNS records; `cloudflared` Windows service on the box ?
`api.khelsutra.guru`; Access apps + email-allowlist policies for `app` & `api`; SPA prod env
50 - [ ] **D3:** deployed smoke E2E (owner-only allowlist); go-live security checklist
(HOSTING_PLAN $7) all green
51 - [ ] **GATE GD:** checklist signed off
52
53   ### Phase R ? rollout
54 - [ ] Wave 1: 5 testers (C13/camera-pilot contacts) + feedback channel (WhatsApp group
or Google Form)
55 - [ ] 2-week retro ? punch list ? widen toward 25 ? beta planning (UI_PROPOSAL $4)
56 - Metrics: matches/week · % rallies corrected · median turnaround · verdict mix · week-2
tester retention
57
58   ## Working agreement
59   Claude (Cowork) designs, codes, writes tests + docs, and leaves a
`scratch/PRn_HANDOFF.md` per

```

```

60     work item. **Claude Code executes all git/PR mechanics** (branch, commit, push, PR,
conflict
61     fixes, merges after owner review) by reading that handoff ? the owner never copy-pastes
62     commands (ratified 2026-06-11 after PR-1/#88 friction; supersedes the command-block
pattern).
63     Cowork's sandbox cannot run git on mounts (ADR-009) and CI runs the offline suite on
every PR.
64     One PR per item; no stacking; never push to master directly. PowerShell note: no `&&`.
65
66     ## Risks & mitigations
67     Box uptime (offline banner, alpha-acceptable) · Gemini 429 burst-throttle (serial
workers,
68     `ai_max_workers=1`, 6-attempt backoff ? learned 2026-06-10) · CF free-tier 100 MB body
cap
69     (chunked upload design) · single-GPU box (alpha default = Gemini path; WASB jobs queue-
depth 1) ·
70     .guru renewal cost ?18.8k/3-yr (optional: transfer to Cloudflare Registrar at-cost after
the
71     60-day registration lock; domain-protection may need toggling for transfer).
72
73     ## Session map (default order)
74     S1?S2 PR-1 · S3 PR-2 · S4 PR-3 · S5 PR-4 · S6 M1+M2 · S7 M3 · S8 M4 · S9 L3 E2E + fixes
·
75     S10 D1?D3 · S11 wave-1 launch
76

```

20. user (2026-06-20T08:00:56.073Z)

```

418     def get_video(self, video_id: str) -> Optional[Dict[str, Any]]:
419         with self._connect() as conn:
420             row = conn.cursor().execute("SELECT * FROM videos WHERE id = ?",
(video_id,)).fetchone()
421             if row:
422                 d = dict(row)
423                 d["validation_results"] = json.loads(d["validation_results"] or "[]")
424                 return d
425             return None
426
427     def query_play_segments(self, video_id: str, filters: Dict[str, Any]) ->
List[Dict[str, Any]]:
428         """
429         Dynamically queries play segments based on filters:
430         - duration_min: float
431         - server: string
432         - receiver: string
433         - event_type: string (e.g. smash, foul)
434         """
435         query = "SELECT r.* FROM play_segments r WHERE r.video_id = ?"
436         params = [video_id]
437
438         if filters.get("duration_min"):
439             query += " AND r.duration >= ?"
440             params.append(filters["duration_min"])
441
442         if filters.get("server"):
443             query += " AND r.server = ?"
444             params.append(filters["server"])
445
446         if filters.get("receiver"):
447             query += " AND r.receiver = ?"
448             params.append(filters["receiver"])
449
450         if filters.get("event_type"):
451             # Check if there is an event of a specific type inside this segment
452             query += " AND EXISTS (SELECT 1 FROM events e WHERE e.segment_id = r.id AND

```

```

e.type = ?)"
453         params.append(filters["event_type"])
454
455     segments_list = []
456     with self._connect() as conn:
457         cursor = conn.cursor()
458         rows = cursor.execute(query, params).fetchall()
459
460         for row in rows:
461             r_dict = dict(row)
462             r_id = r_dict["id"]
463             r_dict["metadata"] = json.loads(r_dict["metadata"] or "{}")
464
465             # Fetch associated events
466             event_rows = cursor.execute("SELECT * FROM events WHERE segment_id = ?",
(r_id,)).fetchall()
467             r_dict["events"] = []
468             for er in event_rows:
469                 e_dict = dict(er)
470                 e_dict["details"] = json.loads(e_dict["details"] or "{}")
471                 r_dict["events"].append(e_dict)
472
473             segments_list.append(r_dict)
474
475     return segments_list
476
477 def clear_video_data(self, video_id: str):
478     with self._connect() as conn:
479         conn.cursor().execute("DELETE FROM videos WHERE id = ?", (video_id,))
480         conn.commit()
481
482 # --- Async job store (DEFERRED_HARDENING_PLAN #16) -----
483 # Writers follow the repo convention (swallow + logger.error, return bool); the
484 # worker logs loudly on a failed transition so a job can't silently wedge.
485
486 def create_job(self, job_id: str, video_path: str, segmenter: Optional[str] = None,
487               player_pool: Optional[List[str]] = None,
488               max_frames: Optional[int] = None,
489               policy: Optional[Dict[str, Any]] = None) -> bool:
490     """Insert a new job in state 'queued'. ``policy`` is an optional JSON
ExecutionPolicy
491     (EXECUTION_POLICY.md) the worker honours; None ? the worker's default policy."""
492     return self._execute_write(
493         "INSERT INTO jobs (id, video_path, segmenter, player_pool, max_frames,
status, policy) "
494         "VALUES (?, ?, ?, ?, ?, 'queued', ?)",
495         (job_id, video_path, segmenter,
496          json.dumps(player_pool) if player_pool is not None else None,
497          max_frames, json.dumps(policy) if policy is not None else None),
498         f"Failed to create job {job_id}",
499     )
500
501 def mark_job_running(self, job_id: str) -> bool:
502     return self._execute_write(
503         "UPDATE jobs SET status='running', started_at=CURRENT_TIMESTAMP, "
504         "progress='starting' WHERE id = ?", (job_id,),
505         f"Failed to mark job {job_id} running",
506     )
507
508 def set_job_progress(self, job_id: str, progress: str) -> bool:
509     return self._execute_write(
510         "UPDATE jobs SET progress=? WHERE id = ?", (progress, job_id),
511         f"Failed to set progress for job {job_id}",
512     )
513

```

```

514         def set_job_video_id(self, job_id: str, video_id: str) -> bool:
515             return self._execute_write(
516                 "UPDATE jobs SET video_id=? WHERE id = ?", (video_id, job_id),
517                 f"Failed to set video_id for job {job_id}",
518             )
519
520         def mark_job_done(self, job_id: str, result: Dict[str, Any]) -> bool:
521             """Job EXECUTED to completion. The pipeline's own verdict (success/failed
522             validation) is inside `result` ? job status 'done' means 'the worker
523             finished'."""
524             return self._execute_write(
525                 "UPDATE jobs SET status='done', progress='finished', result=?, "
526                 "finished_at=CURRENT_TIMESTAMP WHERE id = ?",
527                 (json.dumps(result), job_id),
528                 f"Failed to mark job {job_id} done",

```

21. assistant (2026-06-20T08:01:05.173Z)

I now have a complete picture. Let me verify there's no `/api/health` endpoint (HOSTING_PLAN mentions one as a ping target) and check the static frontend to see what the existing UI actually consumes ? to confirm whether a "list videos" or "list rallies as a standalone GET" endpoint exists.

22. user (2026-06-20T08:01:06.886Z)

docs\UI_PROPOSAL.md:59: | B6 | ****Correction-loop endpoints**** | `PATCH /api/segments/{id}` ? accept/reject/boundary-nudge persisted to `candidate\_segments.human\_label`/`notes` (+ provenance: who, when). This is v1-scope "correction loop" made real. |
docs\UI_ALPHA_EXECUTION_PLAN.md:33:- [] ****PR-4 · B6 correction loop:**** `PATCH /api/segments/{id}` accept/reject/boundary-nudge ? `candidate\_segments.human\_label`/`notes` + provenance (who/when); retire `backend/api/static` (B5)
docs\HOSTING_PLAN.md:42: offline" banner (a `/api/health` ping). Beta removes this with the queue.

23. user (2026-06-20T08:01:07.611Z)

```

backend\api\static\index.html
backend\api\static\style.css
backend\api\static\app.js

```

24. user (2026-06-20T08:01:08.043Z)

No matches found

25. assistant (2026-06-20T08:01:11.851Z)

Confirmed: no `/api/health`, no list-videos endpoint, no `get\_all\_videos` DB method. Let me quickly scan the existing static `app.js` to confirm how the current UI gets segments (it must use POST /api/query), which proves there's no GET-rallies endpoint.

26. user (2026-06-20T08:01:13.271Z)

```

87:         // Async job model (#16): POST returns 202 + job_id; poll /api/jobs/{id}.
88:         const response = await fetch("/api/process", {
108:             const jr = await fetch(`/api/jobs/${jobId}`);
329:         const res = await fetch("/api/query", {
373:         const res = await fetch("/api/compile", {
392:         const dUrl =
`/api/highlights/${currentVideoId}/${encodeURIComponent(outputName)}`;

```

27. assistant (2026-06-20T08:01:21.363Z)

Confirmed. The existing UI gets segments via `POST /api/query` (line 329), not a GET. I now have everything needed to produce the complete API map, auth model, and gaps analysis.

Key findings confirmed:

- 11 endpoints total across `backend/main.py` (7) and `backend/api/uploads.py` (4).
- **Zero auth** anywhere ? no `Depends`, no `HTTPBearer`, no token/email check in any handler. `user\_id` columns exist in the DB schema but are not read or written by any endpoint.
- Intended auth (HOSTING_PLAN §2 lines 37-38) = Cloudflare Access `Cf-Access-Authenticated-User-Email` header; intended local-dev fallback (`DEV\_USER\_EMAIL`) is documented in UI_ALPHA_EXECUTION_PLAN PR-3 (line 32) but **not yet built**.
- For the per-rally-selection UI: no list-videos, no GET-rallies-as-JSON, no per-rally thumbnail/frame, no video/proxy serve, and no correction endpoint. Custom selection ? reel IS possible via `POST /api/compile` (it takes arbitrary `segment\_ids`), but rally listing is only available via `POST /api/query`.

28. user (2026-06-20T08:02:06.890Z)

Structured output provided successfully

29. assistant (2026-06-20T08:02:15.547Z)

API map complete. Summary of what I found:

11 endpoints, all in `backend/main.py` (7) and `backend/api/uploads.py` (4). `job\_worker.py` has no routes (internal worker only). Only one router (`uploads\_api.router`) is included.

Auth model: NONE today. No `Depends`/`HTTPBearer`/token/`DEV\_USER\_EMAIL` check in any handler. CORS is wide open. The DB has `user\_id` columns (migration v4/v5) but no endpoint reads or writes them. Intended auth = Cloudflare Access `Cf-Access-Authenticated-User-Email` header for alpha (HOSTING_PLAN.md:37-38), with a `DEV\_USER\_EMAIL` local fallback scoped to the **unstarted PR-3** (UI_ALPHA_EXECUTION_PLAN.md:32).

Biggest gaps for an interactive per-rally-selection UI:

- No way to list videos/matches at all (no `GET /api/videos`, no `get\_all\_videos` DB method ? you must already know the `video\_id`).
- Rallies are only listable via `POST /api/query` (backend/main.py:331), not a GET resource; no per-rally thumbnail/frame, no source-video/proxy stream for seeking.
- Custom-selection ? reel **does exist** (`POST /api/compile`, backend/main.py:342, takes arbitrary `segment\_ids`) but returns a server filepath, not a download URL.
- No correction endpoint (`PATCH /api/segments/{id}` is the unstarted PR-4).
- No `/api/health` despite HOSTING_PLAN expecting it for the offline banner.

Full structured detail returned via StructuredOutput.